C++ 模板 (Templates) 面试突击指南 (C++11/14 终极版)

针对找工作时间紧张且目标技术栈为 **C++11/14** 的情况，请重点掌握以下 6 个维度的内容。这份清单融合了基础语法与 C++11 核心机制，覆盖了 95% 以上的模板面试题。

1. 核心基础与参数类型

1.1 函数模板与自动推导

编译器会根据传入的参数**自动推导**模板类型。

```
template <typename T>
T add(T a, T b) { return a + b; }
// 使用: add(1, 2); (推导为int) / add<double>(1.5, 2); (显式指定)
```

1.2 类模板

在 C++11/14 中，类模板在实例化时**必须显式指定类型**。

```
template <typename T>
class MyClass { public: T value; };
// 使用: MyClass<int> obj;
```

1.3 非类型模板参数 (Non-type Template Parameters)

模板参数不仅可以是类型，还可以是**编译期常量**（如整型、枚举、指针等）。

```
template <typename T, int N>
class Array {
private:
    T data[N]; // N 在编译期决定了数组大小
};
// 使用: Array<int, 10> arr;
```

2. 模板特化与重载决议 (面试必考)

2.1 全特化与偏特化

- **全特化**：为特定类型（如 `char*` ）提供完全不同的实现。
- **偏特化**：限制部分参数（如限制为指针 `T*` 或数组 `T[]` ）。 ⚠️ **核心陷阱：函数模板不允许偏特化！只能全特化或重载。类模板既可以全特化也可以偏特化。**

2.2 模板重载匹配规则 (优先级)

当普通函数、模板函数、特化模板同时存在时，编译器的调用优先级是： **普通函数 > 特化模板函数 > 普通模板函数**

```
void func(int x) {}          // 1. 普通函数 (最高优)
template<> void func(int x) {} // 2. 特化模板
template<typename T> void func(T x) {} // 3. 普通模板
// func(10) 会优先调用 1
```

3. C++11 模板的灵魂：完美转发 (T0级别考点)

这是 C++11 引入的最重要的模板机制，面试官一定会问！

3.1 万能引用 (Universal / Forwarding Reference)

当 `T` 是模板参数时， `T&&` **不是右值引用，而是万能引用**。它可以接收左值，也可以接收右值。

```
template<typename T>
void func(T&& arg) { ... } // arg 可以绑定到 10(右值)，也可以绑定到变量 x(左值)
```

3.2 引用折叠规则 (Reference Collapsing)

当推导出的类型出现引用的引用时，C++11 的折叠规则如下（口诀：**只要有左值引用，结果就是左值引用**）：

- `T& &` -> `T&`
- `T& &&` -> `T&`
- `T&& &` -> `T&`
- `T&& &&` -> `T&&`

3.3 完美转发 (`std::forward`)

作用：在模板函数中，将参数透传给内部函数，并**完美保持其原本的左值/右值属性**。

```
template<typename T>
void wrapper(T&& arg) {
    // 如果不加 std::forward，arg 作为有名变量，传递给 func 时都会变成左值！
    func(std::forward<T>(arg));
}
```

4. 经典面试题/踩坑点

4.1 为什么模板的声明和实现通常放在同一个头文件（.h/.hpp）中？

必答要点：C++采用**独立编译模型**。模板只有在实例化时才生成真正的代码。如果实现放在 `.cpp` 中，其他文件调用时找不到实例化后的实现代码，会导致**链接错误 (undefined reference)**。

4.2 `typename` 的特殊用法

用于指明**依赖于模板参数的嵌套类型**，否则编译器会把它当成静态成员变量。

```
template <typename T>
void print(T& obj) {
    typename T::iterator it = obj.begin(); // 必须加 typename
}
```

5. 现代 C++ 模板进阶 (C++11 / C++14)

5.1 可变参数模板 (Variadic Templates)

用于接收任意数量的参数。C++11 面试常考**递归解包法**。

```
#include <iostream>
void print() { std::cout << std::endl; } // 1. 递归终止函数

template <typename T, typename... Args>
void print(T head, Args... rest) {       // 2. 展开函数
    std::cout << head << " ";
    print(rest...); // 递归调用，每次剥离一个参数
}
```

5.2 SFINAE 与 Type Traits (类型萃取)

SFINAE (替换失败并非错误)。配合 `<type_traits>` 在编译期限制类型。**C++14 引入了 `_t` 和 `_v` 后缀**。

```
#include <type_traits>
// C++14 极简写法：只有当 T 是整型时才参与编译
template <typename T, std::enable_if_t<std::is_integral<T>::value, int> = 0>
void process(T t) { }
```

5.3 变量模板与自动推导 (C++14 特性)

- **变量模板：** `template<typename T> constexpr T pi = T(3.1415);`
- `decltype(auto)`：用于模板函数返回值，完美保留返回值的引用和 `const` 属性。

```
template<typename Container>
decltype(auto) getElement(Container& c, int index) {
    return c[index]; // 完美返回元素的引用，允许修改 c[index] = 10;
}
```

💡 总结复习策略：

1. **第一梯队（默写级）：完美转发**（万能引用 `T&&` + 引用折叠 + `std::forward`），这是 C++11 的半壁江山；**重载匹配优先级**；手写普通类/函数模板。
2. **第二梯队（精通级）**：弄懂特化与偏特化（牢记函数模板无偏特化）；手撕**可变参数模板的递归展开**。
3. **第三梯队（原理与秀操作级）**：SFINAE（`std::enable_if`）、非类型模板参数、解释为什么模板要放头文件。